Predicción de volatilidad financiera

Apuntes teóricos y computacionales para la Semana 6

Volatilidad histórica mediante ventanas móviles

Objetivo de la semana. A partir de la base de rendimientos financieros construida en la Semana 5, calcular medidas de volatilidad histórica mediante ventanas móviles de 5, 10, 20 y 30 días. Se estudiará la interpretación matemática de la desviación estándar móvil, el efecto del tamaño de la ventana sobre la sensibilidad de la medida, la anualización de la volatilidad y las validaciones necesarias para garantizar que los cálculos utilicen exclusivamente información disponible hasta cada fecha.

Índice

1. Contexto de la Semana 6 **4**

2. Objetivos de aprendizaje **4**

3. De la magnitud de un movimiento a la volatilidad **5**

4. Volatilidad histórica **5**

5. La idea de una ventana móvil **6**

6. El orden temporal es fundamental **6**

7. Ventanas de 5, 10, 20 y 30 días **7**

8. Ejemplo manual **7**

9. Desviación estándar muestral y parámetro `ddof` **8**

10. Lectura de la base de rendimientos **8**

11. Validaciones de entrada **9**

12. Cálculo de volatilidad histórica en Python **10**

13. Valores faltantes inducidos por las ventanas **11**

14. Verificación manual del primer valor de `Vol_5` **12**

15. Verificación en una fecha intermedia **12**

16. Efecto del tamaño de la ventana **13**

17. Volatilidad diaria y volatilidad anualizada **13**

18. Estadística descriptiva de la volatilidad **14**

19. Visualización conjunta de las cuatro ventanas **15**

20. Comparación entre `Vol_5` y `Vol_30` **16**

21. Rendimientos absolutos y volatilidad histórica **16**

22. Identificación de periodos de mayor volatilidad **17**

23. Comparación relativa de corto y largo plazo **18**

24. Validaciones automáticas **18**

24.1. No negatividad 18

24.2. Valores finitos 19

24.3. Número correcto de valores faltantes 19

24.4. Consistencia con una ventana manual 19

25. Construcción de la base final de la semana **19**

26. Verificación del archivo exportado **20**

27. Ejercicios **21**

28. Checklist de cierre **21**

29. Producto esperado **22**

1. Contexto de la Semana 6

En la Semana 5 se transformó la base limpia de precios de `SPY` en una base de rendimientos financieros. El archivo de entrada para esta semana es

`data/processed/SPY_2015_2025_returns.csv`.

La base contiene, entre otras variables,

`Return`, `Abs_Return`, `Sq_Return`,

donde

$$r_t = \log\left(\frac{P_t}{P_{t-1}}\right), \qquad |r_t|, \qquad r_t^2.$$

Estas transformaciones describen el movimiento observado en un día particular. Sin embargo, la volatilidad intenta resumir la intensidad de las fluctuaciones durante un periodo de varios días. El siguiente paso natural consiste, por tanto, en estudiar la dispersión de los rendimientos dentro de ventanas temporales móviles.

La secuencia de trabajo de esta semana será

base de rendimientos $\longrightarrow$ ventanas móviles $\longrightarrow$ volatilidad histórica $\longrightarrow$ comparación de escalas temporales.

Delimitación de la semana. En esta etapa se calculará únicamente volatilidad histórica retrospectiva. Todavía no se construirán rezagos ni indicadores predictivos, no se definirá la volatilidad futura que funcionará como variable objetivo y no se implementarán modelos EWMA o GARCH. Estos elementos corresponden a semanas posteriores del proyecto.

2. Objetivos de aprendizaje

Al finalizar la semana, la estudiante deberá ser capaz de:

1. explicar qué se entiende por volatilidad histórica;
2. distinguir entre un rendimiento diario y una medida de volatilidad;
3. interpretar una ventana móvil de tamaño w;
4. calcular la media móvil de los rendimientos dentro de una ventana;
5. calcular la desviación estándar muestral dentro de una ventana móvil;
6. construir volatilidades históricas de 5, 10, 20 y 30 días;
7. explicar el efecto del tamaño de ventana sobre la sensibilidad de la volatilidad;

8. identificar correctamente los valores faltantes inducidos por las ventanas móviles;
9. verificar manualmente un cálculo realizado con `rolling()`;
10. distinguir entre volatilidad diaria y volatilidad anualizada;
11. visualizar y comparar medidas de volatilidad construidas con diferentes ventanas;
12. localizar periodos de alta volatilidad;
13. validar automáticamente no negatividad, finitud y consistencia de los cálculos;
14. preservar correctamente el orden temporal de la información;
15. exportar la base procesada que servirá como entrada para la Semana 7.

3. De la magnitud de un movimiento a la volatilidad

En la Semana 5 se utilizaron $|r_t|$ y r_t^2 para describir la magnitud de un movimiento diario. Estas cantidades son útiles, pero corresponden a una sola fecha.

Por ejemplo, si

$$r_t = -0.04,$$

entonces

$$|r_t| = 0.04, \qquad r_t^2 = 0.0016.$$

Estas transformaciones indican que el movimiento del día t fue relativamente grande, pero no responden directamente a preguntas como:

- ¿han sido inusualmente variables los últimos cinco días?;
- ¿la variabilidad reciente es mayor que la observada durante el último mes?;
- ¿un movimiento extremo fue aislado o forma parte de un periodo prolongado de inestabilidad?

Para responder este tipo de preguntas necesitamos resumir varios rendimientos consecutivos. La desviación estándar es una de las medidas más sencillas para hacerlo.

4. Volatilidad histórica

Sea una serie de rendimientos

$$r_1, r_2, \ldots, r_T.$$

Para una ventana de longitud w, consideramos en la fecha t los últimos w rendimientos disponibles:

$$r_{t-w+1}, r_{t-w+2}, \ldots, r_t.$$

La media de esos rendimientos es

$$\bar{r}_{t,w} = \frac{1}{w} \sum_{i=t-w+1}^{t} r_i.$$

La volatilidad histórica de ventana w se define mediante la desviación estándar muestral:

$$\sigma_{t,w} = \sqrt{\frac{1}{w-1} \sum_{i=t-w+1}^{t} (r_i - \bar{r}_{t,w})^2}.$$

Esta cantidad es siempre no negativa:

$$\sigma_{t,w} \geq 0.$$

Si los rendimientos dentro de la ventana son muy parecidos entre sí, la volatilidad será pequeña. Si presentan variaciones grandes, la volatilidad será mayor.

Interpretación. La volatilidad histórica no indica si el mercado subió o bajó. Mide dispersión. Tanto una sucesión de grandes rendimientos positivos como una sucesión de grandes rendimientos negativos pueden producir una volatilidad elevada.

5. La idea de una ventana móvil

Una ventana móvil se desplaza una observación a la vez a lo largo de la serie temporal.

Para $w = 5$, la primera ventana utiliza

$$r_1, r_2, r_3, r_4, r_5,$$

y produce $\sigma_{5,5}$.

La siguiente utiliza

$$r_2, r_3, r_4, r_5, r_6,$$

y produce $\sigma_{6,5}$.

Después,

$$r_3, r_4, r_5, r_6, r_7$$

produce $\sigma_{7,5}$, y así sucesivamente.

Por tanto, dos ventanas consecutivas comparten casi todas sus observaciones. Esta es una de las razones por las que la volatilidad histórica cambia de forma relativamente suave, especialmente cuando w es grande.

6. El orden temporal es fundamental

La volatilidad calculada en la fecha t debe utilizar solamente información disponible hasta esa fecha. Para una ventana de longitud w se emplean

$$r_{t-w+1}, \ldots, r_{t-1}, r_t.$$

No deben utilizarse

$$r_{t+1}, r_{t+2}, \ldots$$

para construir una variable disponible en el tiempo t.

Esta diferencia será particularmente importante cuando el proyecto pase de descripción a predicción.

Principio temporal. Las volatilidades de esta semana son variables retrospectivas. La volatilidad $\sigma_{t,w}$ utiliza el rendimiento de la fecha t y rendimientos anteriores. La volatilidad futura, que se construirá posteriormente como variable objetivo, utilizará rendimientos posteriores a t. No deben confundirse ambas definiciones.

7. Ventanas de 5, 10, 20 y 30 días

Durante esta semana se construirán cuatro medidas:

$$\sigma_{t,5}, \qquad \sigma_{t,10}, \qquad \sigma_{t,20}, \qquad \sigma_{t,30}.$$

En la base de datos se almacenarán como

`Vol_5`, `Vol_10`, `Vol_20`, `Vol_30`.

Cada ventana representa una escala temporal distinta:

Variable	Ventana	Comportamiento esperado
`Vol_5`	5 días	Muy sensible a movimientos recientes
`Vol_10`	10 días	Sensibilidad intermedia
`Vol_20`	20 días	Más estable
`Vol_30`	30 días	Más suave y persistente

Una ventana pequeña responde rápidamente cuando aparece un rendimiento extremo. Sin embargo, también puede disminuir rápidamente cuando esa observación abandona la ventana.

Una ventana grande incorpora más historia. Por ello suele reaccionar de forma más lenta, pero también conserva durante más tiempo la influencia de periodos turbulentos.

8. Ejemplo manual

Considérese una secuencia hipotética de cinco rendimientos diarios:

$$0.010, \quad -0.005, \quad 0.012, \quad -0.008, \quad 0.006.$$

La media es

$$\bar{r} = \frac{0.010 - 0.005 + 0.012 - 0.008 + 0.006}{5} = 0.003.$$

La varianza muestral es

$$s^2 = \frac{1}{4}\sum_{i=1}^{5}(r_i - 0.003)^2.$$

Por tanto,

$$s = \sqrt{\frac{1}{4}\sum_{i=1}^{5}(r_i - 0.003)^2}.$$

Esta cantidad sería la volatilidad histórica de cinco días correspondiente al último rendimiento de la ventana.

El ejemplo muestra que la volatilidad no se obtiene simplemente tomando el promedio de $|r_t|$. Se calcula como una medida de dispersión alrededor de la media local.

9. Desviación estándar muestral y parámetro `ddof`

La fórmula utilizada en el proyecto contiene el denominador $w - 1$:

$$\sigma_{t,w} = \sqrt{\frac{1}{w-1}\sum_{i=t-w+1}^{t}(r_i - \bar{r}_{t,w})^2}.$$

En `pandas`, la función `std()` utiliza por defecto la desviación estándar muestral, equivalente a `ddof=1`. No obstante, en este proyecto se escribirá explícitamente

```
.std(ddof=1)
```

para que el código refleje de forma directa la definición matemática.

Si se utilizara `ddof=0`, el denominador sería w y se obtendría otra convención de varianza. La diferencia es pequeña para ventanas grandes, pero conviene mantener una definición consistente durante todo el proyecto.

10. Lectura de la base de rendimientos

El notebook de esta semana puede llamarse

`06_volatilidad_historica.ipynb`.

Se utilizarán las bibliotecas

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
```

Las rutas principales serán

```
ruta_processed = Path("data/processed")

archivo_returns = (
    ruta_processed / "SPY_2015_2025_returns.csv"
)

archivo_volatility = (
    ruta_processed / "SPY_2015_2025_volatility.csv"
)
```

Antes de leer la base se verifica que exista:

```
if not archivo_returns.exists():
    raise FileNotFoundError(
        f"No se encontro el archivo: {archivo_returns}"
    )
```

La lectura se realiza mediante

```
datos_returns = pd.read_csv(
    archivo_returns,
    index_col=0,
    parse_dates=True
)

datos = datos_returns.copy(deep=True)
```

Después conviene revisar

```
print(datos.shape)
print(datos.index.min())
print(datos.index.max())
print(datos.columns.tolist())
datos.head()
```

11. Validaciones de entrada

Antes de construir nuevas variables, se verifican algunas condiciones básicas de la base recibida de la Semana 5.

```
columnas_necesarias = [
    "Adj Close",
    "Return",
    "Abs_Return",
    "Sq_Return"
]

faltantes = [
    col for col in columnas_necesarias
```

```
    if col not in datos.columns
]

assert len(faltantes) == 0, (
    f"Faltan columnas: {faltantes}"
)
```

También debe preservarse el orden temporal:

```
assert datos.index.is_monotonic_increasing
assert datos.index.is_unique
```

La base final de la Semana 5 no debería tener valores faltantes en los rendimientos:

```
assert not datos["Return"].isna().any()
assert not datos["Abs_Return"].isna().any()
assert not datos["Sq_Return"].isna().any()
```

Finalmente:

```
assert np.isfinite(datos["Return"]).all()
```

Estas pruebas permiten detectar inmediatamente si el archivo de entrada no corresponde a la versión esperada del pipeline.

12. Cálculo de volatilidad histórica en Python

Definimos las cuatro ventanas de trabajo:

```
ventanas = [5, 10, 20, 30]
```

Después construimos cada columna mediante

```
for w in ventanas:
    datos[f"Vol_{w}"] = (
        datos["Return"]
        .rolling(window=w)
        .std(ddof=1)
    )
```

El argumento `window=w` especifica el número de observaciones utilizadas. Como la ventana queda alineada con la observación final, el valor almacenado en la fecha t corresponde a los rendimientos que terminan precisamente en t.

Podemos inspeccionar las primeras filas mediante

```
columnas_vol = [
    "Return",
    "Vol_5",
    "Vol_10",
```

```
    "Vol_20",
    "Vol_30"
]

datos[columnas_vol].head(35)
```

13. Valores faltantes inducidos por las ventanas

Para calcular una desviación estándar de cinco observaciones se necesitan cinco rendimientos. En consecuencia, las primeras cuatro fechas no tienen todavía una ventana completa.

De manera general, una ventana de tamaño w produce

$$w - 1$$

valores faltantes iniciales cuando se exige una ventana completa.

Por tanto, en esta semana esperamos:

$$\begin{aligned} \texttt{Vol_5}: &\quad 4, \\ \texttt{Vol_10}: &\quad 9, \\ \texttt{Vol_20}: &\quad 19, \\ \texttt{Vol_30}: &\quad 29. \end{aligned}$$

Podemos comprobarlo con

```
print(
    datos[
        ["Vol_5", "Vol_10", "Vol_20", "Vol_30"]
    ].isna().sum()
)
```

Y convertir la expectativa matemática en una prueba automática:

```
esperados = {
    "Vol_5": 4,
    "Vol_10": 9,
    "Vol_20": 19,
    "Vol_30": 29
}

for variable, n_nan in esperados.items():
    assert datos[variable].isna().sum() == n_nan
```

Interpretación correcta. Estos valores `NaN` no representan errores ni observaciones perdidas. Son consecuencia de no disponer todavía de suficientes rendimientos anteriores para completar cada ventana.

14. Verificación manual del primer valor de `Vol_5`

Una comprobación especialmente útil consiste en reproducir manualmente el primer valor válido de la ventana de cinco días.

Los primeros cinco rendimientos de la base son

```
primeros_cinco = datos["Return"].iloc[:5]
print(primeros_cinco)
```

La desviación estándar muestral se calcula con

```
vol_manual = primeros_cinco.std(ddof=1)
```

El primer valor válido producido por la ventana móvil debe aparecer en la quinta fila, es decir, en la posición 4:

```
vol_rolling = datos["Vol_5"].iloc[4]

print("Calculo manual:", vol_manual)
print("Rolling:", vol_rolling)
```

Finalmente:

```
assert np.isclose(
    vol_manual,
    vol_rolling
)
```

Esta prueba confirma que `rolling(5).std(ddof=1)` está implementando exactamente la definición utilizada en los apuntes.

15. Verificación en una fecha intermedia

No conviene validar únicamente el primer valor. Podemos seleccionar una posición intermedia, por ejemplo $i = 100$, y reconstruir la ventana de 20 días.

```
w = 20
i = 100

ventana_manual = (
    datos["Return"]
    .iloc[i-w+1:i+1]
)

vol_manual = ventana_manual.std(ddof=1)
vol_python = datos["Vol_20"].iloc[i]

print("Inicio:", ventana_manual.index[0])
print("Fin:", ventana_manual.index[-1])
```

```
print("Observaciones:", len(ventana_manual))
print("Manual:", vol_manual)
print("Python:", vol_python)

assert len(ventana_manual) == w
assert np.isclose(vol_manual, vol_python)
```

Esta comprobación también permite visualizar con claridad qué observaciones participan en $\sigma_{t,20}$.

16. Efecto del tamaño de la ventana

Las cuatro medidas de volatilidad no deben interpretarse como estimaciones independientes de fenómenos completamente distintos. Todas resumen los mismos rendimientos, pero utilizan diferentes cantidades de historia.

Una ventana corta, por ejemplo $w = 5$:

- reacciona rápidamente a cambios recientes;
- puede presentar picos más pronunciados;
- también puede disminuir rápidamente;
- es más sensible a una observación individual.

Una ventana larga, por ejemplo $w = 30$:

- utiliza más información pasada;
- produce una trayectoria generalmente más suave;
- responde más lentamente a cambios repentinos;
- mantiene durante más tiempo el efecto de periodos turbulentos.

No existe una ventana universalmente correcta. En el proyecto se conservarán varias escalas porque posteriormente podrán aportar información complementaria a los modelos predictivos.

17. Volatilidad diaria y volatilidad anualizada

Las columnas `Vol_5`, `Vol_10`, `Vol_20` y `Vol_30` están expresadas en la misma escala temporal que los rendimientos diarios.

En aplicaciones financieras es común expresar la volatilidad en escala anual. Bajo la aproximación usual de independencia temporal y varianza proporcional al tiempo, se utiliza

$$\sigma_{\text{anual}} = \sigma_{\text{diaria}}\sqrt{252},$$

donde 252 representa aproximadamente el número de sesiones bursátiles en un año.

Por ejemplo, si

$$\sigma_{\text{diaria}} = 0.01,$$

entonces

$$\sigma_{\text{anual}} \approx 0.01\sqrt{252} \approx 0.159,$$

es decir, aproximadamente 15.9 %.

Podemos construir versiones anualizadas mediante

```
for w in ventanas:
    datos[f"Vol_{w}_Annual"] = (
        datos[f"Vol_{w}"] * np.sqrt(252)
    )
```

> **Convención del proyecto.** Las variables principales se conservarán en escala diaria para mantener coherencia con los rendimientos y con la variable objetivo que se definirá posteriormente. La anualización se utilizará como herramienta de interpretación, no como sustituto de las columnas básicas de volatilidad.

18. Estadística descriptiva de la volatilidad

Una vez construidas las cuatro series puede obtenerse un resumen mediante

```
variables_vol = [
    "Vol_5",
    "Vol_10",
    "Vol_20",
    "Vol_30"
]

resumen_vol = (
    datos[variables_vol]
    .describe()
    .T
)

resumen_vol
```

La tabla permite comparar:

- número de observaciones válidas;
- media;
- desviación estándar;
- mínimo;
- cuartiles;

- máximo.

El número de observaciones no será exactamente igual para las cuatro columnas, porque cada ventana induce una cantidad distinta de valores faltantes al inicio de la serie.

También puede calcularse la matriz de correlación:

```
correlaciones = datos[variables_vol].corr()
correlaciones
```

Se espera una asociación positiva importante entre las diferentes ventanas porque todas se construyen a partir de la misma serie de rendimientos. Sin embargo, correlación alta no significa que sean variables idénticas.

19. Visualización conjunta de las cuatro ventanas

La comparación principal de esta semana puede construirse con

```
fig, ax = plt.subplots(figsize=(12, 5))

for w in ventanas:
    ax.plot(
        datos.index,
        datos[f"Vol_{w}"],
        label=f"{w} dias",
        linewidth=1
    )

ax.set_title(
    "Volatilidad historica de SPY"
)
ax.set_xlabel("Fecha")
ax.set_ylabel("Volatilidad")
ax.legend()
plt.show()
```

La gráfica debe utilizarse para responder cualitativamente:

- ¿qué ventana presenta los cambios más abruptos?;
- ¿cuál produce la trayectoria más suave?;
- ¿los máximos ocurren aproximadamente en los mismos periodos?;
- ¿la volatilidad disminuye con la misma rapidez en todas las escalas?

20. Comparación entre `Vol_5` y `Vol_30`

Para apreciar con mayor claridad el efecto de la longitud de la ventana, conviene comparar únicamente los extremos:

```
fig, ax = plt.subplots(figsize=(12, 5))

ax.plot(
    datos.index,
    datos["Vol_5"],
    label="5 dias",
    linewidth=1
)

ax.plot(
    datos.index,
    datos["Vol_30"],
    label="30 dias",
    linewidth=1.2
)

ax.set_title(
    "Comparacion entre ventanas de 5 y 30 dias"
)
ax.set_xlabel("Fecha")
ax.set_ylabel("Volatilidad")
ax.legend()
plt.show()
```

Esta figura permite observar directamente la relación entre rapidez de reacción y suavizamiento.

21. Rendimientos absolutos y volatilidad histórica

Una conexión natural con la Semana 5 consiste en comparar $|r_t|$ con una medida de volatilidad de varios días.

```
fig, ax = plt.subplots(figsize=(12, 4))

ax.plot(
    datos.index,
    datos["Abs_Return"],
    linewidth=0.7,
    label="|Return|"
)

ax.plot(
    datos.index,
    datos["Vol_20"],
    linewidth=1.2,
    label="Volatilidad 20 dias"
```

```
)

ax.set_title(
    "Magnitud de rendimientos y volatilidad historica"
)
ax.set_xlabel("Fecha")
ax.set_ylabel("Magnitud")
ax.legend()
plt.show()
```

El rendimiento absoluto corresponde a una sola observación:

$$|r_t|.$$

La volatilidad de 20 días resume un conjunto de observaciones:

$$\sigma_{t,20}.$$

Por ello, un día extremo puede producir un pico inmediato en $|r_t|$, mientras que su efecto sobre $\sigma_{t,20}$ puede permanecer durante varias fechas.

22. Identificación de periodos de mayor volatilidad

En la Semana 5 se localizaron días con movimientos extremos. Ahora interesa identificar fechas pertenecientes a periodos de alta variabilidad.

Por ejemplo, podemos ordenar la base por `Vol_20`:

```
mayor_volatilidad = (
    datos[
        [
            "Adj Close",
            "Return",
            "Vol_5",
            "Vol_10",
            "Vol_20",
            "Vol_30"
        ]
    ]
    .dropna()
    .nlargest(10, "Vol_20")
)

mayor_volatilidad
```

Esta tabla ya no responde a la pregunta

¿cuáles fueron los diez días con mayor movimiento?

sino a una pregunta distinta:

¿en qué fechas la variabilidad de los últimos 20 días alcanzó sus niveles más altos?

Diferencia conceptual. Un rendimiento extremo es un evento diario. Una volatilidad elevada describe un periodo. Un día puede presentar un movimiento grande sin que necesariamente exista un episodio prolongado de alta volatilidad.

23. Comparación relativa de corto y largo plazo

Como ejercicio exploratorio puede construirse el cociente

$$Q_t = \frac{\sigma_{t,5}}{\sigma_{t,30}}.$$

En Python:

```
datos["Ratio_Vol_5_30"] = (
    datos["Vol_5"] /
    datos["Vol_30"]
)
```

Cuando

$$Q_t > 1,$$

la variabilidad de los últimos cinco días es mayor que la variabilidad estimada con los últimos treinta días. Esto puede ser indicativo de un incremento reciente de la intensidad de los movimientos.

Cuando

$$Q_t < 1,$$

la volatilidad de corto plazo es menor que la medida de horizonte más largo.

Esta variable se introduce solamente con fines de interpretación. Su posible uso como predictor pertenece a la construcción formal de variables de la Semana 7.

24. Validaciones automáticas

Las medidas de volatilidad admiten varias comprobaciones útiles.

24.1. No negatividad

Por definición,

$$\sigma_{t,w} \geq 0.$$

Por tanto:

```python
for w in ventanas:
    valores = datos[f"Vol_{w}"].dropna()
    assert (valores >= 0).all()
```

24.2. Valores finitos

```python
for w in ventanas:
    valores = datos[f"Vol_{w}"].dropna()
    assert np.isfinite(valores).all()
```

24.3. Número correcto de valores faltantes

```python
for w in ventanas:
    assert (
        datos[f"Vol_{w}"].isna().sum()
        == w - 1
    )
```

24.4. Consistencia con una ventana manual

```python
w = 20
i = 100

esperado = (
    datos["Return"]
    .iloc[i-w+1:i+1]
    .std(ddof=1)
)

obtenido = datos[f"Vol_{w}"].iloc[i]

assert np.isclose(esperado, obtenido)
```

Estas pruebas convierten propiedades matemáticas de las definiciones en verificaciones automáticas del pipeline.

25. Construcción de la base final de la semana

Las variables principales que se incorporarán de manera permanente son

`Vol_5`, `Vol_10`, `Vol_20`, `Vol_30`.

La base final puede construirse mediante

```
columnas_finales = [
    "Open",
    "High",
    "Low",
    "Close",
    "Adj Close",
    "Volume",
    "Return",
    "Abs_Return",
    "Sq_Return",
    "Vol_5",
    "Vol_10",
    "Vol_20",
    "Vol_30"
]

datos_vol = datos[columnas_finales].copy()
```

En esta etapa **no se eliminarán** automáticamente las primeras 29 observaciones. Los valores faltantes iniciales de las volatilidades son estructurales y conservarlos permite documentar correctamente cómo se construyó cada variable.

La exportación se realiza mediante

```
datos_vol.to_csv(
    archivo_volatility,
    index=True
)
```

La estructura del pipeline queda entonces

$$\texttt{raw} \longrightarrow \texttt{clean} \longrightarrow \texttt{returns} \longrightarrow \texttt{volatility}.$$

26. Verificación del archivo exportado

Conviene volver a leer el archivo para comprobar que la exportación preservó su estructura:

```
verificacion = pd.read_csv(
    archivo_volatility,
    index_col=0,
    parse_dates=True
)

assert verificacion.shape == datos_vol.shape
assert verificacion.index.is_monotonic_increasing
assert verificacion.index.is_unique
```

También se verifican las columnas nuevas:

```
for w in ventanas:
```

```
    columna = f"Vol_{w}"
    assert columna in verificacion.columns
    assert verificacion[columna].isna().sum() == w - 1
```

La presencia de estos `NaN` iniciales es esperada y debe preservarse hasta que se construya, en semanas posteriores, la matriz definitiva utilizada para modelado.

27. Ejercicios

1. Obtenga estadísticas descriptivas de las cuatro medidas de volatilidad `Vol_5`, `Vol_10`, `Vol_20` y `Vol_30` y compare sus medias, desviaciones estándar y valores máximos.
2. Grafique conjuntamente `Vol_5`, `Vol_10`, `Vol_20` y `Vol_30`. Describa el efecto del tamaño de la ventana.
3. Identifique las diez fechas con mayor `Vol_20`. Compare esas fechas con los mayores valores de `Abs_Return` obtenidos en la Semana 5. ¿Son exactamente las mismas observaciones?
4. Calcule la matriz de correlación entre las cuatro volatilidades. Explique por qué pueden estar altamente correlacionadas sin ser idénticas.

28. Checklist de cierre

Antes de considerar terminada la Semana 6, debe verificarse que:

- la base de entrada es `SPY_2015_2025_returns.csv`;
- la serie conserva un índice temporal ordenado y sin duplicados;
- `Return` no contiene valores faltantes al iniciar la semana;
- se construyeron `Vol_5`, `Vol_10`, `Vol_20` y `Vol_30`;
- todas las volatilidades válidas son no negativas y finitas;
- cada ventana contiene exactamente el número esperado de `NaN` iniciales;
- se verificó manualmente al menos un valor de volatilidad;
- se comprendió la diferencia entre ventanas cortas y largas;
- se construyó al menos una versión anualizada con el factor $\sqrt{252}$;
- se obtuvieron estadísticas descriptivas de las cuatro ventanas;
- se generó la gráfica conjunta de volatilidades;
- se compararon específicamente las ventanas de 5 y 30 días;
- se identificaron periodos de mayor volatilidad;
- no se utilizaron observaciones futuras en ninguna variable de esta semana;

- el archivo `SPY_2015_2025_volatility.csv` fue exportado y verificado.

29. Producto esperado

Al finalizar la semana se deberán tener los siguientes productos:

1. notebook `06_volatilidad_historica.ipynb`;
2. base `data/processed/SPY_2015_2025_volatility.csv`;
3. columnas de volatilidad histórica de 5, 10, 20 y 30 días;
4. tabla descriptiva de las cuatro medidas de volatilidad;
5. matriz de correlación entre las diferentes ventanas;
6. gráfica conjunta de volatilidad histórica;
7. tabla con las diez fechas de mayor `Vol_20`;

Conexión con la Semana 7. La base construida aquí contiene medidas de volatilidad en diferentes escalas temporales. En la siguiente semana se construirán variables rezagadas e indicadores simples utilizando exclusivamente información disponible hasta cada fecha. Estas variables servirán posteriormente como entradas para los modelos de predicción.